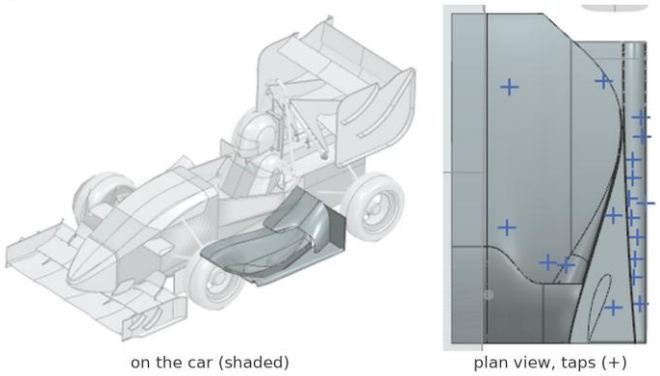
Where to Put the Pressure Sensors: Sparse Reconstruction of a Formula SAE Side-Diffuser Pressure Field with the SVD

Muhammad Abdullahi | Student ID 34977473 | Department of Mechanical and Aerospace Engineering, Monash University |
https://github.com/mabd0073-coder/sparse-sensor-placement

Abstract — A racing car's floor makes downforce by creating low pressure underneath it. Monash Motorsport wants to measure this pressure on its Formula SAE car with a few dozen sensors, yet know it over the whole side diffuser, so that its simulations (CFD) can be checked against the real car. I learn the typical pressure patterns from up to 122 CFD simulations with the singular value decomposition (SVD), choose sensor positions with a greedy D-optimal search, and rebuild the whole field from the sensor readings by least squares. The real data need 13 to 27 patterns, not the single pattern an earlier, flawed dataset suggested; that dataset mixed millimetres and metres and its results are withdrawn. With 10 sensors, fewer than the number of patterns, no method does much better than predicting the average field. With 20 sensors, the channels one working box of the team's pressure scanner leaves after reference pressures, spread at least 0.05 m apart, the error for unseen conditions roughly halves on the better-mapped ensemble, including cornering conditions the model never saw (0.22 against 0.45 in pressure coefficient). The error falls steeply until there are as many sensors as patterns (about 16) and levels off near 0.2 beyond about 20, so the 38 taps first planned would gain little; reading noise of the scanner's size barely matters. Restricted to drillable points, 20 taps keep most of the accuracy in leave-one-out but lose most of it on cornering cases unless cornering simulations are part of the design set. The same CFD cases mapped onto a different reference mesh give no clear gain, so the way separately meshed simulations are combined matters as much as where the sensors go. Greedy search and QR pivoting crowd sensors within millimetres of each other, and a minimum spacing improved accuracy in every test. I also redesigned the simulation plan with space-filling sweeps. Trained on the 16 new cornering cases that returned in time, 20 sensors rebuild 13 withheld cases, 12 of them straight-line, to 0.29 against 0.56 with no sensors.

Index Terms — proper orthogonal decomposition, singular value decomposition, sparse sensor placement, D-optimal design, pressure coefficient, Formula SAE, side diffuser, leave-one-out validation.



1. Engineering problem

Section 1.1 gives the background that a reader outside vehicle aerodynamics needs; Appendix A is a glossary.

1.1 Background

Why downforce matters. A tyre can only push sideways or lengthways about as hard as it is pressed onto the road. Downforce presses the car onto the road without adding mass, so the car can corner, brake and accelerate harder. Like other aerodynamic forces it grows with the square of speed (Katz, 2006). The team's lap simulation ranks the downforce coefficient as the third most influential car parameter for competition points (Thomas, Hazon and Deva Doss, n.d.).

Where it comes from. Much of a racing car's downforce comes from its underside. Air squeezed through the narrow gap between the floor and the ground speeds up, and by Bernoulli's principle faster air has lower pressure, so the floor is sucked towards the ground; this is *ground effect* (Zhang, Toet and Zerihan, 2006). At the rear the gap widens. This *diffuser* slows the air back down so that it rejoins the surrounding air smoothly, which lets the air under the floor run faster. The **side diffusers** studied here are the floor sections either side of the driver, between the front and rear wheels (Figure 1a).

The car's attitude (Figure 1; Milliken and Milliken, 1995). The **front and rear ride heights** (FRH, RRH) are the floor-to-ground gaps near the front and rear axles, in millimetres. If they differ the car is **pitched**; braking, for example, pushes the nose down. In a corner the body **rolls** outwards, the front wheels are **steered**, and the car points at a small **yaw** angle to its direction of travel. The **corner radius** R sets the sideways acceleration V^2/R, which is 1.1 to 1.4 g in the team's cornering simulations. Each of these changes the air gap or the direction of the air, and so changes the pressure on the diffuser.

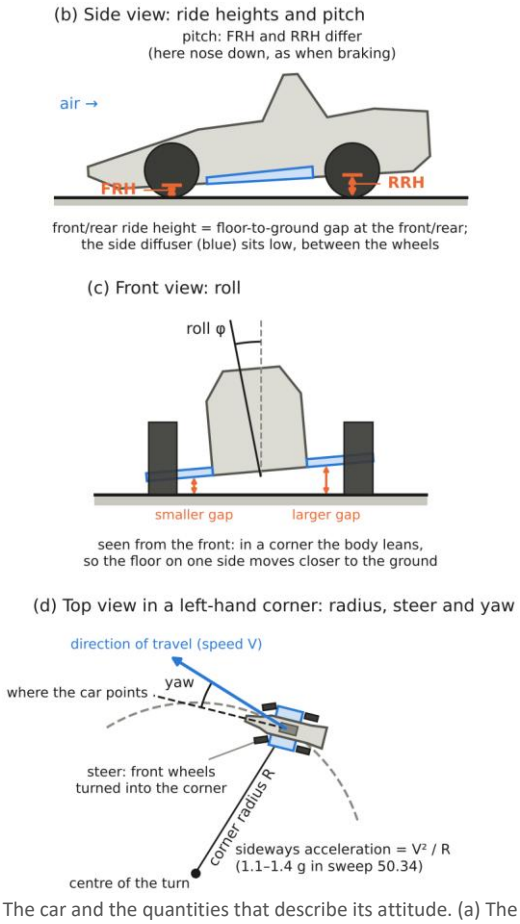


Figure 1. The car and the quantities that describe its attitude. (a) The side diffuser in the team's CAD model: on the car (shaded) and in plan view with

the planned tap positions (+) imported from the drillable layout. (b)–(d) Schematic, not to scale; angles exaggerated. Sensors go on the left-hand side diffuser only (y < 0).

Pressure coefficient. Pressure is reported as Cp = (p − p∞)/(½ρV²), where p is the local pressure, p∞ the pressure of the undisturbed air, ρ the air density and V the car's speed. Dividing by ½ρV² removes most of the effect of speed; in wind-tunnel tests of the team's car the downforce coefficient changed by less than 7 % between 11 and 19 m/s (Sayer, n.d.). Cp = 0 means the surrounding pressure, +1 means the air has been brought to rest, and negative values mean suction (Figure 3).

A simple model. Figure 2 uses an idealised model of the gap under a floor, in which the air speed is set by the gap size and the pressure by Bernoulli's equation. It shows the two effects that matter most here: a lower floor and a nose-down pitch both give more suction. The model ignores friction and flow separation, which make real floors lose downforce below a critical ride height (Zhang et al., 2006); that is one reason detailed CFD is needed.

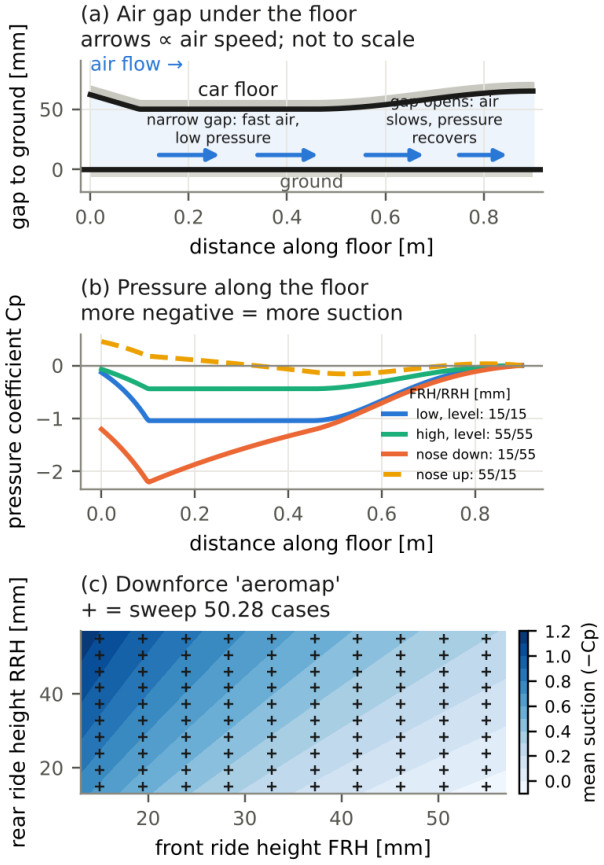


Figure 2. Idealised floor-and-diffuser model (illustrative, not the team's geometry; `experiments/toy_diffuser_demo.py`). (a) The air gap; arrows show air speed. (b) Cp along the floor for four attitudes. (c) Mean suction for every front/rear ride-height pair; + marks the 100 conditions simulated by the team (sweep 50.28).

The CFD data. The team simulates the airflow with ANSYS Fluent on Monash's M3 computing cluster. One simulation (a **case**) takes about 10 hours and gives Cp at about 145,000 points on the side-diffuser surface (Figure 3). Straight-line cases (SYM) simulate half the car with a mirror plane on the centreline; cornering cases use a rotating reference frame (RRF), so that the air approaches along curved paths. A set of cases run together is a **sweep**.

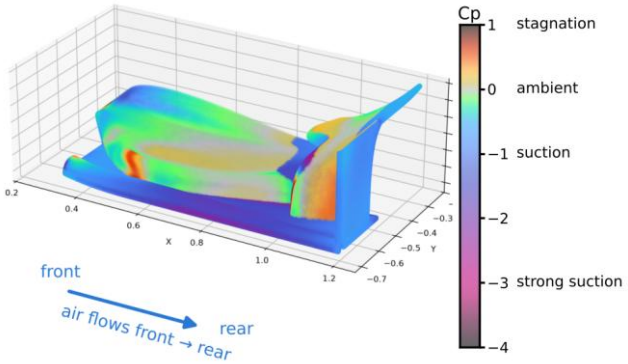


Figure 3. One real CFD case (case 0 of sweep 50.28): Cp on the left-hand side diffuser, seen from outside the car and slightly above. The diffuser has a low floor plate, a curved upper surface and a vertical plate at the rear; suction is strongest on the floor plate, nearest the ground. Image from `analysis.py`, cropped. On the colour bar, Cp = −1 means air moving about 1.4 times the car speed and −3 about twice it (Bernoulli, away from the surface).

1.2 The problem and why it matters

To check the CFD against the real car (*correlation*), the team will fit pressure taps: small holes in the surface connected by tubes to a pressure scanner. Each tap needs tubing and a scanner channel, so only tens can be fitted, against about 145,000 CFD values. The problem is: **given M CFD cases, choose k sensor positions (k ≈ 10 to 40) from which the full Cp field of a new, unseen condition can be rebuilt.** The answer tells the team where to drill. The team already has the hardware: a differential pressure measurement system (DPMS) of three 24-channel transducer boxes, used in the wind tunnel on the front and rear wings (Thomas et al., n.d.). Only the left-hand diffuser (y ≤ 0) will be instrumented, and the taps must avoid internal structure, which is marked by exclusion zones taken from the CAD model. The approach grew from a prototype by Lachlan Sayer (Monash Motorsport) that used one CFD case plus artificial noise (Sayer, 2026), itself based on Inoba et al. (2022) and Manohar et al. (2018); this project develops it into a full pipeline on real multi-case CFD data, with validation, comparisons and performance analysis.

1.3 Scope and limitations

The study uses CFD only, so each rebuilt field is checked against the CFD field of the same case rather than against measurements. Readings are treated as exact except in a noise study, tubing lag and sensor size are ignored, and the results apply only to conditions similar to those simulated.

2. Inputs, outputs and data

Table 1 lists the top-level inputs and outputs. Function-level details are in the docstrings and the HTML documentation (`docs/html/sensor_opt.html`).

Table 1. Top-level inputs and outputs (functions in `sensor_opt/core.py`).

Item	Format, units and range	Meaning and handling of bad input
In: `side_diffuser.csv`, one per case	Text table: x, y, z (mm, converted to m); pressures (Pa); Cp (–); speed (m/s). About 145,000 rows.	Missing or empty file, or NaN/infinite values, raises `CfdDataError`.
In: case parameters	From the case folder name: FRH, RRH (mm); roll, steer, yaw (deg); corner radius (m); speed (m/s).	Parsed by pattern matching (a capital-letter key followed by a number); a parameter missing from the name is left out.
In: candidate points, exclusion flag	`all_points.csv` from CAD (mm): 702,040 points, 146,503 after removing duplicates.	Points within 30 mm of internal structure (`CAD/point_sorter.py`) excluded; a mask of the wrong length is ignored with a warning.
In: settings	Variance target 0.90; k sensors; $\varepsilon = 10^{-10}$; 4 interpolation neighbours; sensors on $y \le 0$.	`ValueError` if more sensors than candidate points are requested.
Out: modes U_r, singular values σ	N × r array (dimensionless); r values.	The pressure patterns and how much each matters.
Out: sensor positions	k point indices and (x, y, z) in metres.	Where to drill the pressure taps.
Out: rebuilt field, RMS error	Cp at all N points; error in Cp units.	Compared with the CFD field of the same case.

Data. Sweep 50.28 (SYM, half car) varies FRH and RRH independently over a 10 × 10 grid from 15 to 55 mm in straight-line running at 19.44 m/s. Sweep 50.34 (RRF) is a cornering sweep at FRH 31.5 mm and RRH 32 mm in which steer and speed are tied to the corner radius (4 to 18 m; 6.5 to 15.5 m/s) while roll and yaw stay zero, so it is a single line through the space of conditions. Of its 29 cases, 22 (radius 5.5 to 18 m) produced results. The other seven stopped during set-up, were cancelled, or ended without writing output; the cornering (RRF) workflow is custom-built and not yet as robust as the team's main CFD workflow. The raw 50.34 surface files have since been deleted, so these cases exist only in the two combined files B1 and B2. Cp is taken from Fluent's output; across the 22 cornering cases the speed changes from 7.6 to 15.5 m/s (V^2 four times larger), yet the mean Cp stays between −0.37 and −0.41, so Cp is normalised by each case's own speed and the two sweeps are comparable.

Table 2 lists the four ensembles built from these sweeps. They differ in how the separately meshed cases were mapped onto one set of points (Section 3.2), and this turned out to decide the results (Section 4.4). Only the left-hand side ($y \le 0$) is used in every ensemble.

Table 2. Ensembles. All are built from the same CFD cases; only the mapping differs.

Name	Cases and mapping	Points (y ≤ 0) and status
A0	50.28 (100); interpolation onto the CAD point grid	146,503. Withdrawn: grid in mm, CFD points in m, so fields are nearly constant (Section 4.4)
A	50.28 (100); interpolation (IDW, 4 points) onto the densest case mesh	144,472. Main straight-line ensemble
B1	50.28 + 50.34 (122); nearest neighbour onto a whole-car mesh	141,679 of 283,700
B2	50.28 + 50.34 (122); nearest neighbour onto a left-side case mesh	143,709

3. Computational approach

3.1 The idea in plain terms

The pressure fields of different cases share a few typical patterns, such as "more suction everywhere" when the car runs lower. If every field is the average field plus a mix of r patterns, then r numbers (the amount of each pattern) describe a case, much as three numbers for red, green and blue describe any colour on a screen. A few well-placed sensors can measure those r amounts, and the full field follows (Figure 4). Figures 5a to 5d show each step on the toy model of Figure 2, using 100 cases on the same ride-height grid as sweep 50.28: (a) the cases; (b) the same data as a matrix with one column per case, so the cases are kept side by side and never averaged together; (c) the patterns found by the SVD, the first carrying 98.5 % of the variation; (d) an unseen case rebuilt from two greedily chosen sensors, with an RMS error of 0.001 in Cp against 0.44 for guessing the average field.

Figure 4. Workflow from CFD output to a validated sensor layout.

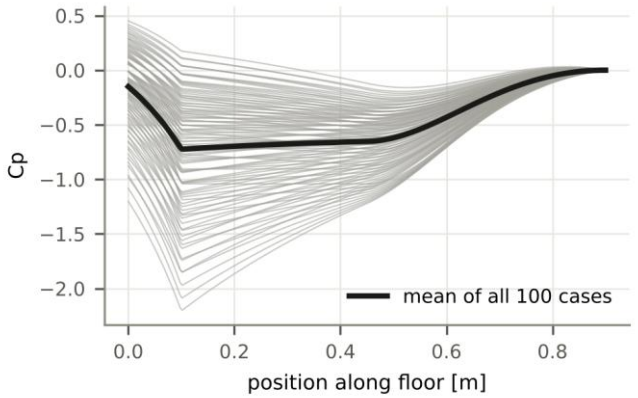


Figure 5a. The toy model of Figure 2 run for 100 cases (FRH and RRH 15 to 55 mm): one Cp curve per case; black is the mean field.

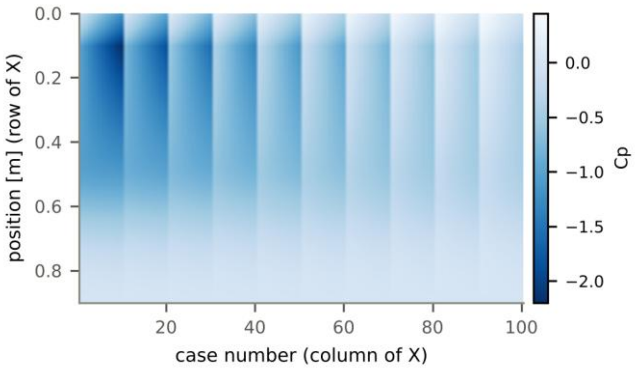


Figure 5b. The same data as the snapshot matrix X: one column per case, one row per position. Cases are kept side by side, never averaged together.

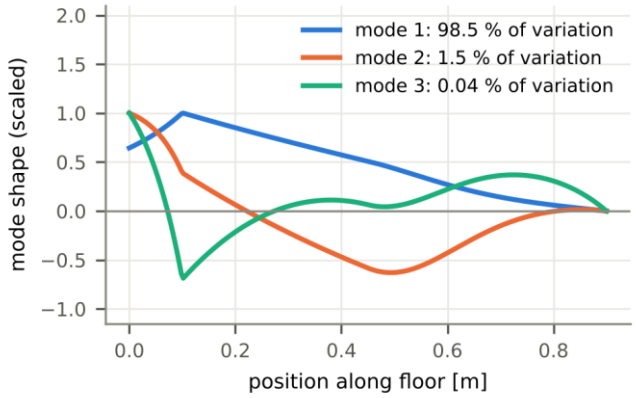


Figure 5c. The first three patterns (modes) found by the SVD of X with the mean subtracted, with the share of the variation each carries.

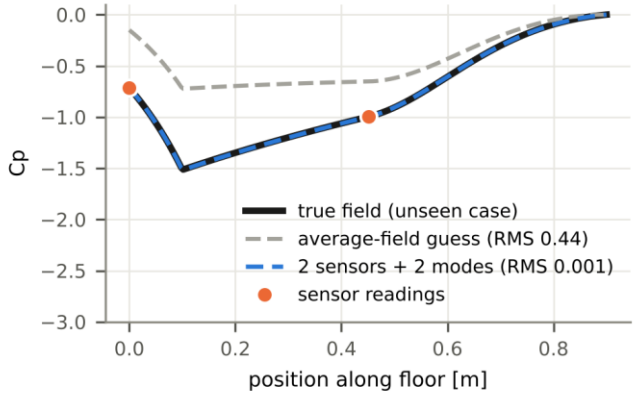


Figure 5d. An unseen case (FRH 22 mm, RRH 48 mm) rebuilt from two greedily chosen sensors and two modes, against the average-field guess. Figures 5a to 5d are produced by experiments/toy_diffuser_demo.py.

3.2 Stacking the cases: mesh alignment

Each case becomes one column of the snapshot matrix X (N points × M cases). Every row must refer to the same physical point, but Fluent meshes every case differently, so each case is mapped onto one reference point set. Ensemble A uses inverse-distance weighting of the 4 nearest points (Shepard, 1968), a local interpolation method in the sense of Week 3 of the unit (Monash University, 2026) onto the densest case mesh; B1 and B2 copy the value of the nearest point, which is quicker but produces stripes where mesh panels meet. The units of both point sets must match: Ensemble A0 interpolated CFD points in metres onto a grid in millimetres, which is why it was withdrawn. Half-car cases were meant to be mirrored across the centreline when combined with whole-car cases, but the mirroring condition never triggered (Section 4.4); it is now fixed and tested, and all results here use the left side only, where no mirroring is needed. Figure 6 shows the difference: IDW blends neighbouring values, which smooths sharp suction peaks and, near a thin plate, can mix the two faces, while nearest-neighbour keeps true values but in steps. This is one possible reason why results differ between ensembles; it has not been checked on the real meshes.

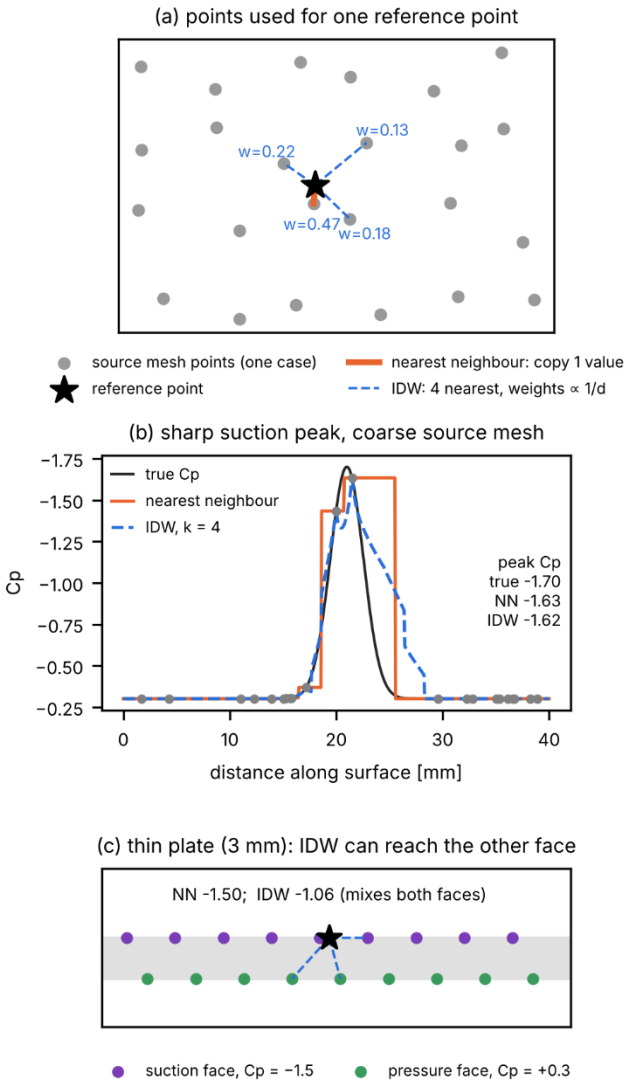


Figure 6. Mapping one case's mesh onto a reference point (synthetic illustration). (a) Nearest-neighbour (k = 1) copies the closest value; IDW averages the 4 nearest with weights ∝ 1/d, as in `cluster/master_load_sweep_ensemble.py`. (b) On a coarse mesh nearest-neighbour gives steps and IDW smears a suction peak downstream. (c) Near a 3 mm plate the 4 nearest points include the opposite face, so IDW gives −1.06 where the true value is −1.50.

3.3 Finding the patterns: POD with the SVD

The average field $\bar{x}$ is subtracted from each column and the SVD is taken; this is proper orthogonal decomposition (POD) computed with the SVD (Sirovich, 1987; Berkooz et al., 1993):

$$X - \bar{x}1^T = U\,\Sigma\,V^T$$

$$r = \min\{\, r : \Sigma_{i \le r}\,\sigma_i^2 \,/\, \Sigma_i\,\sigma_i^2 \ge 0.90 \,\}$$

The columns of U are the patterns (modes), the singular values σ_i measure how much variation each carries, and V holds the amount of each pattern in each case. Keeping the r modes that carry 90 % of the variation gives $x \approx \bar{x} + U_r a$, with r numbers a per case. Because $N \gg M$, the thin SVD costs $O(NM^2)$ operations.

3.4 Choosing sensors: greedy D-optimal selection

Readings at k points give k equations for the r unknowns a. The greedy D-optimal method adds sensors one at a time, each time taking the point that most increases the determinant of the information matrix $A = \varepsilon I + \Sigma_j u_j u_j^T$, where u_j is row j of U_r, which minimises the effect of independent sensor noise on a.

Adding point u multiplies det A by $1 + u^T A^{-1} u$ (the matrix determinant lemma), and A^{-1} is updated by the Sherman–Morrison formula (Sherman and Morrison, 1950) instead of being re-inverted:

$$p_k = \text{argmax}_i \, (1 + u_i{}^T \, A^{-1} \, u_i)$$
$$A^{-1} \leftarrow A^{-1} - (A^{-1}u)(A^{-1}u)^T \, / \, (1 + u^T A^{-1} u)$$

Candidates are restricted to $y \le 0$ and to points outside the exclusion zones. Greedy selection is a standard, fast heuristic (Joshi and Boyd, 2009; Manohar et al., 2018) but is not guaranteed to find the best set. Because it chooses the points where the patterns are largest, it tends to put sensors next to each other, so an optional minimum spacing (`min_dist`, 0.05 m here) removes candidates closer than that to a chosen sensor (*spaced greedy*). While $k \le r$ and ε is tiny, the gain is dominated by the part of u not explained by the sensors already chosen, so the greedy method picks the same points as QR pivoting; the two agree exactly in the 10-sensor results below.

3.5 Rebuilding the field; settings

$$a = \text{argmin} \, \| \, U_s \, a - (y - \bar{x}_s) \, \|_2 \, , \quad \hat{x} = \bar{x} + U_r \, a$$

U_s holds the rows of U_r at the sensors, y the readings and $\bar{x}_s$ the average field there; with $k \ge r$ the least-squares problem is well posed, and with $k < r$ it is rank deficient and has no unique solution (Monash University, 2026, Notebook 2.3). Implementation: NumPy and SciPy (Harris et al., 2020; Virtanen et al., 2020). Settings: variance target 0.90; $\varepsilon = 10^{-10}$, which makes A invertible at the start; 4 interpolation neighbours; 4 to 80 sensors; minimum spacing 0.05 m; 30 mm exclusion distance and 15 mm edge margin (Section 5.2).

3.6 Why this method; assumptions

With about 145,000 values per field but only about 100 cases, a low-rank linear model suits the data. It has few parameters, its patterns can be inspected, and it rests on linear algebra taught in the unit, whereas a neural network would have far more parameters than there are cases. The method assumes that (i) a new case is close to a mix of the training patterns, (ii) the flow is steady and the CFD represents the car, and (iii) the readings are exact. Its main weakness is features that *move*, such as a sharp pressure change at an edge that shifts with ride height.

4. Validation

4.1 Verification: is the code correct?

The core functions (`sensor_opt/core.py`) were tested on problems with known answers, using synthetic fields (Table 3). A field built from r known modes is rebuilt to machine precision for r = 1, 3 and 8, with exactly r modes found; with noisy readings the error falls as sensors are added; and the vectorised greedy search picks exactly the same sensors as the original loop. Further tests check that spaced greedy keeps sensors at least `min_dist` apart and reproduces the unconstrained picks when `min_dist` = 0, that invalid input raises a clear error, and that the loaders mirror one-sided data from either side but leave whole-car data alone (this test fails on the old mirroring code). All 20 pytest tests pass (`tests/`, `docs/test_report.txt`). As a regression check, `analysis.py` was re-run after moving its functions into the module and reproduced every previous sensor choice and both RMS errors (0.158962 and 0.195828) exactly.

Table 3. Verification on synthetic fields.

Check (synthetic data)	Result
Exact recovery, true rank 1, 3 and 8 (N = 20,000, M = 40)	Max error 8.9×10^{-16} (machine precision)
Noise σ = 0.05, rank 3, 4 / 8 / 16 / 32 sensors	RMS 0.0162 / 0.0118 / 0.0081 / 0.0062
Vectorised vs loop greedy search	Identical sensor lists
Spaced greedy, min_dist 0.05 m	All pairs ≥ 0.05 m apart; min_dist 0 gives the original picks

4.2 Leave-one-out validation

The main test treats each case in turn as unseen. The average, modes and sensors are rebuilt from the other cases; the hidden case is "measured" at those sensors only, rebuilt, and compared with its own CFD field by RMS error. The hidden case never influences the model, so this estimates the error for a new condition from the same family. It is always compared with a baseline that uses no sensors and predicts the training-average field, the baseline comparison recommended for regression models (Monash University, 2026, Notebook 5.5).

On Ensemble A (100 straight-line cases), 15 or 16 modes are needed for 90 % of the variation. With 10 greedy sensors the mean leave-one-out error is **0.470** in Cp (s.d. 0.110; worst 0.701) against **0.508** for the baseline, an improvement of only 1.1 times. This is expected: 10 readings cannot fix 15 unknown pattern amounts, so least squares has no unique answer and the rebuilt field stays close to the average. The ride-height trend in the data is clear (the average Cp of a case correlates with FRH + RRH at −0.91: a lower car gives more suction), but the cases also differ a great deal point by point (median standard deviation across cases 0.34 in Cp, up to 3.9 at some points), which may partly be an artefact of mapping thin parts (Section 4.4). Training on ride heights up to 50 mm and testing on the 36 cases above gives 0.679 against 0.701 for the baseline: no useful extrapolation.

4.3 How many sensors?

Figure 7 repeats the leave-one-out test and the straight-to-cornering test on Ensemble B1 for 4 to 80 spaced-greedy sensors (`experiments/sensor_count_ensemble.py`). Below the number of modes r the error stays near the baseline, because there are fewer equations than unknowns. It then falls steeply: in the cornering test from 0.43 with 6 sensors to 0.22 with 16, then only to 0.19 with 80. Beyond about 20 sensors the error is set by the variation the modes leave out, not by the number of readings.

Real readings are noisy. The team's DPMS has an estimated uncertainty of ±7 Pa (Thomas et al., n.d.), about 0.03 in Cp at 19.4 m/s. Adding random noise of that size (10 trials per case) raised the error by less than 0.001 at every sensor count, because least squares averages independent errors and the model error is much larger. At the cornering speeds of sweep 50.34 (6.5 to 15.5 m/s) the same ±7 Pa is about 0.05 to 0.3 in Cp, which was not tested and would matter more. The data therefore point to 16 to 24 taps placed by spaced greedy; more taps buy little until the patterns themselves are better known. This report uses 20: nine of the team's taps are broken and two

of the three DPMS boxes may be out of service, so one 24-channel box with 4 channels kept for reference pressures leaves 20 (Deva Doss, pers. comm., 2026). The 38 taps first planned for the side diffuser would lower the error only from 0.245 to about 0.22 in leave-one-out and from 0.218 to about 0.20 on cornering cases (the values at 32 to 40 sensors).

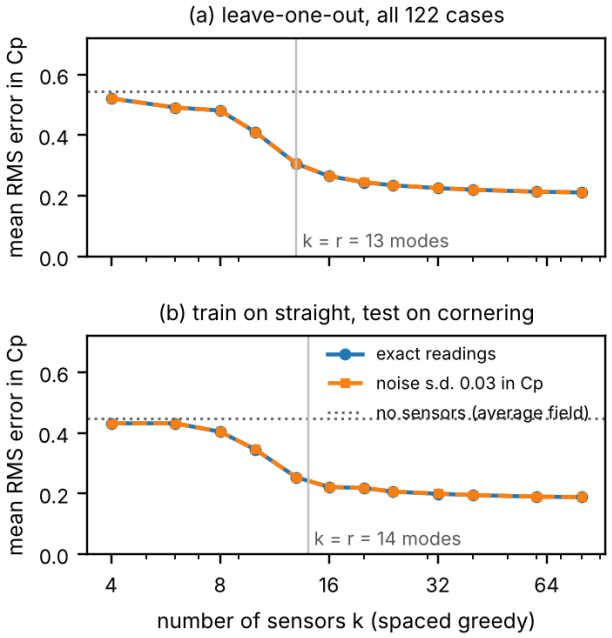


Figure 7. Error against number of sensors on Ensemble B1 (spaced greedy, min_dist 0.05 m). The grey line marks k = r. Noise of 0.03 in Cp (the DPMS uncertainty at 19.4 m/s) changes the curves by less than 0.001.

4.4 Cornering, and why the mapping decides the answer

Withdrawn results. An earlier version of this work reported an error of 0.0036 with one mode, 12.8 times better than the baseline, on Ensemble A0. Comparing A0 with A, which hold the same 100 cases, showed that they contain different fields: case 0 of A0 varies only from −0.597 to −0.585 over the whole surface, and the two average fields correlate at −0.09. The A0 loader interpolated CFD points in metres onto a grid in millimetres, so every grid point drew on the same few CFD points and nearly constant fields resulted; one mode then described them almost perfectly. I cannot re-run that loader because the raw files are gone, so all A0 results are withdrawn. The same review found that the mirroring condition (`is_half_car = y.min() > -0.05`) assumed half-car data at $y \geq 0$, whereas the data lie at $y \leq 0$, so the right half of B1 holds copied edge values; using $y \leq 0$ only avoids this.

Cornering tests. Three tests were run on B1 and B2 at 10 and 20 sensors (`experiments/cornering_test.py`; Table 4, Figure 8): (a) leave-one-out over all 122 cases; (b) training on the 100 straight-line cases and testing on the 22 cornering cases, the hardest test because the model has never seen a corner; and (c) leave-one-out within the 22 cornering cases.

On B1, 20 spaced sensors halve the error in every test, including cornering cases the model never saw: 0.218 against 0.447 for the baseline. Within the cornering cases alone only 3 modes are needed, because steer, speed and radius move together along one line, and the error halves again (0.048 against 0.094); random sensors do as well there (0.045), since almost any 20 points can pin down 3 unknowns. On B2, built from the same CFD cases, the same tests need two to five times as many modes and the sensors give no clear gain: spaced greedy gives 0.232, 0.198 and 0.114 in tests (a) to (c) against 0.276, 0.208 and 0.091 for the baseline. The two ensembles differ only in the reference mesh used for nearest-neighbour mapping, so the mapping between meshes changes the result as much as the sensor method does. Nearest-neighbour copying adds case-to-case jumps where the meshes differ, which appear as extra modes (Figure 9). Spaced greedy is the most reliable method across all tests; plain greedy and QR, which cluster their sensors, are never the best at 20 sensors, and random placement can fail badly (mean 0.8 in test (b) on B2, because a few random points can leave some patterns almost unmeasured).

Table 4. Cornering tests, mean RMS error in Cp (20 sensors; modes kept in brackets)..

Test, ensemble	Spaced greedy	Greedy / QR	No sensors
(a) all 122, B1 (13)	0.245	0.248 / 0.284	0.546
(a) all 122, B2 (27)	0.232	0.354 / 0.354	0.276
(b) straight → corner, B1 (14)	0.218	0.266 / 0.296	0.447
(b) straight → corner, B2 (25)	0.198	0.295 / 0.295	0.208
(c) 22 cornering, B1 (3)	0.048	0.049 / 0.068	0.094
(c) 22 cornering, B2 (14)	0.114	0.129 / 0.176	0.091

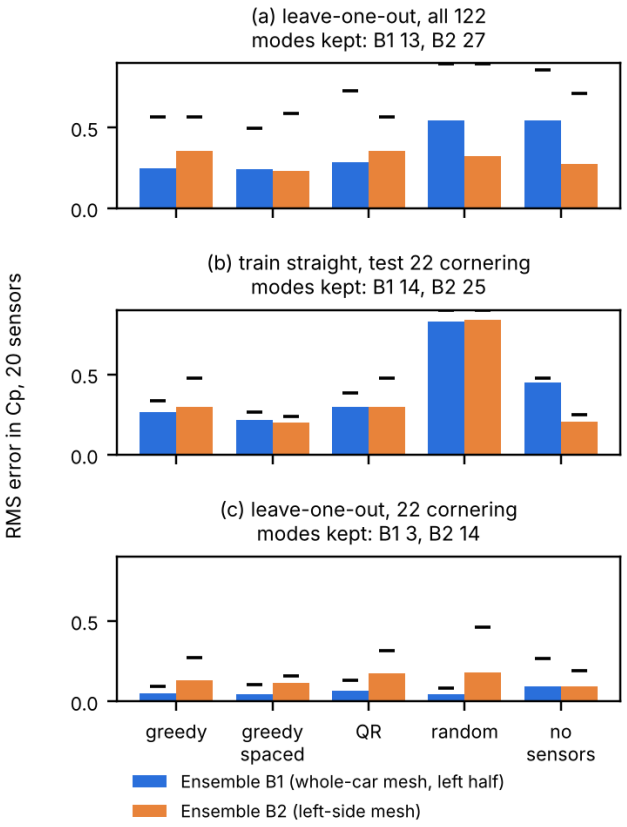


Figure 8. Cornering tests on the two nearest-neighbour ensembles (20 sensors; bar = mean, tick = worst case; values above 0.9 clipped). B1 and B2 hold the same 122 CFD cases mapped onto different reference meshes.

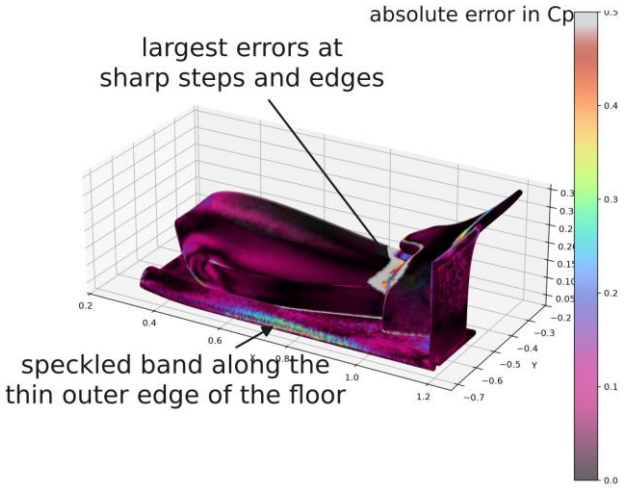


Figure 9. Reconstruction error for case 0 (40 sensors) on a re-aligned ensemble. Bands at panel edges are typical of mapping errors. Image from `analysis.py`, annotated.

4.5 What this validation cannot establish

Agreement with CFD says nothing about agreement with the real car. The team's wind-tunnel correlation found RMS differences of 0.12 to 0.15 in Cp between CFD and front-wing pressure taps (Thomas et al., n.d.). The best reconstruction errors here (about 0.2 for unseen conditions) are larger than that, so at present the sensor layout, not CFD fidelity, would limit on-car correlation. The cornering data cover one ride height, the B ensembles mix two CFD set-ups (SYM and RRF) and use nearest-neighbour mapping, and 22 cases is a small sample. The sensors in Sections 4.2 to 4.4 were not restricted by the exclusion zones; the drillable layout is validated separately in Section 5.2. Neighbouring grid cases are similar, so leave-one-out rounds are not fully independent and the errors are probably optimistic.

5. Comparison and optimisation

5.1 Alternative placement methods

Table 5. Placement methods on Ensemble A, 10 sensors, 100 leave-one-out rounds.

Method	LOO RMS, mean ± s.d.	Worst	Closest pair
Greedy D-optimal	0.470 ± 0.111	0.701	1.7 mm
Greedy, min_dist 0.05 m	0.449 ± 0.111	0.704	52 mm
Greedy, min_dist 0.10 m	0.442 ± 0.112	0.754	101 mm
QR with column pivoting	0.470 ± 0.111	0.701	1.7 mm
Random (20 layouts)	0.486 ± 0.224	4.19	61 mm (mean)
Average field, no sensors	0.508 ± 0.110	0.743	–

QR pivoting is the standard alternative in sparse sensing (Drmač and Gugercin, 2016) and random placement the naive baseline. With 10 sensors and 15 to 16 modes every method stays close to the baseline (Table 5), because there are fewer readings than unknowns; the comparison at 20 sensors is in Table 4. Two results hold in every test. First, greedy and QR choose the same points while $k \le r$ (Section 3.4), and both crowd them together: the closest pair of greedy sensors is 1.7 mm apart. Readings that close carry almost the same information. A minimum spacing of 0.05 to 0.10 m reduces the error by about 5 % here and by up to 34 % at 20 sensors (Table 4), so spaced greedy is the recommended method. Second, random placement is unreliable: its average can be competitive, but some layouts fail badly (worst round 4.19), because they leave some patterns almost unmeasured. Learned regressions are compared with least squares in Section 5.5.

5.2 A layout that can be drilled

The layouts above ignore where a tap can physically go. Plotting the greedy sensors on the surface showed them crowded onto the edge of the floor plate and the tip of a tube, where a tap cannot be fitted. `experiments/feasible_layout.py` therefore restricts the candidates before placement. (i) The CAD exclusion zones (points within 30 mm of internal structure) are transferred from the CAD point grid to the CFD points by nearest neighbour; CFD points with no CAD point within 20 mm are also excluded, because the two geometries do not match exactly. (ii) Sharp edges, corners, open boundaries and thin parts are found from the point cloud: for each point, a principal component analysis of about 30 neighbours spanning 10 to 20 mm gives the *surface variation* $\lambda_{min}/(\lambda_1+\lambda_2+\lambda_3)$, (Pauly, Gross and Kobbelt, 2002), which is near 0 on a smooth surface and large at a crease, and an off-centre neighbourhood marks a boundary. Points within 15 mm of such a feature are excluded. (iii) Sensors must be at least 0.05 m apart. Greedy selection then runs over the remaining 10 % of points (14,505 of 141,679), and an exchange step swaps one sensor at a time for the allowed point that most increases $\log\det(U_s^T U_s)$ (a Fedorov-type exchange; Fedorov, 1972) until no swap helps.

Of the 20 unconstrained greedy sensors, all fail these checks: 15 lie in exclusion zones, the rest on edges, and the closest two are 3.8 mm apart. The feasible layout (Figure 10) keeps every sensor at least 15 mm from an edge and 50 mm from its neighbours, outside all exclusion zones. Rebuilding the layout in every fold on Ensemble B1, the price of feasibility is modest in leave-one-out over all 122 cases (mean RMS 0.258 against 0.245 for unconstrained spaced greedy and 0.546 with no sensors) but large for the unseen cornering cases (0.407 against 0.218; baseline 0.447), because many of the most informative points lie on the excluded edges. With 24 sensors the same test gives 0.315, so the four channels kept for reference pressures cost most of a drillable layout's cornering accuracy. The exchange step raises $\log\det(U_s^T U_s)$ but here worsened the cornering error (0.377 to 0.407), because it optimises for the training modes, not for conditions outside them. When the layout is chosen with the cornering cases included, as it would be for the taps actually drilled, the same 20 taps rebuild those cases to 0.232 even with modes from the straight cases only (Section 5.6), so cornering simulations belong in the design set.

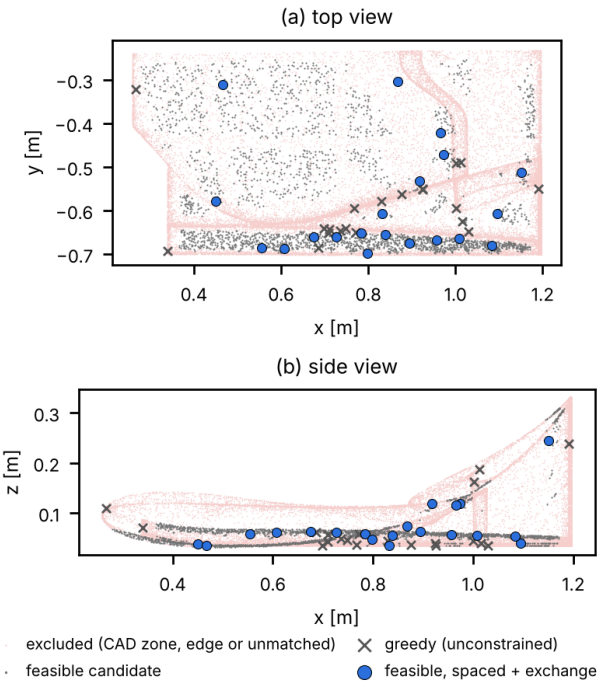


Figure 10. Feasible candidates (dark grey) and excluded regions (pink) on the left side diffuser, with the unconstrained greedy layout (×) and the feasible spaced layout after exchange (blue), 20 sensors on Ensemble B1. Coordinates are in `results/k20/feasible_layout.csv`.

5.3 Performance and optimisation

The original greedy search looped over every candidate in Python. I vectorised it as row sums of $(U_r A^{-1}) \circ U_r$, with a mask for chosen points and the same Sherman–Morrison update; it selects identical sensors (tested). A profile of `analysis.py` with cProfile on Ensemble A (`results/profile_analysis.txt`) shows where the time goes: of 94 s in total, 60 s is spent drawing the 3-D scatter plots in Matplotlib, 4.6 s in the two SVD calls and 2.4 s in the two greedy searches, so the numerical method takes under a tenth of the run.

Table 6. Run time on Ensemble A (N = 144,472, M = 100). Laptop: Intel i7-13650HX, NumPy 2.5.3, best of 3 (`results/timings_laptop.txt`); loop timing from a 2-core cloud machine.

Task	Baseline	Improved
Greedy, 40 sensors, r = 16	12.59 s (Python loop, cloud)	1.35 s laptop (vectorised; 0.45 s cloud, 28× faster)
POD, 144,472 × 100	0.99 s laptop, 1.27 s cloud (thin SVD)	0.25 s cloud (method of snapshots, 5× faster)
Memory, snapshot matrix	115.6 MB	+464 MB peak in SVD

Each greedy step multiplies U_r (N × r) by A^{-1}, about $2Nr^2 = 7.4 \times 10^7$ floating-point operations (FLOP), so 40 sensors cost about 3.0×10^9 FLOP. Each step also streams U_r (18.5 MB) through memory several times, about 1 FLOP per byte, so the search is limited by memory bandwidth rather than arithmetic, in the sense of the roofline model (Monash University, 2026, Notebook 2.1). For the POD, because N ≫ M, the modes can be found from the small M × M matrix X^TX (the *method of snapshots*; Sirovich, 1987): its eigenvectors V give $U_r = XV/\sigma$. On the same machine this is five times faster (1.27 s to 0.25 s), with modes that agree with the SVD to 10^{-14}; it made the 122-round leave-one-out of Section 4.4 practical. The method of snapshots squares the condition number κ (Monash University, 2026, Notebook 9.1), but only the leading modes, which are well separated, are kept. Leave-one-out repeats the POD and the search in every round, multiplying the cost by M.

5.4 Optimising the data: redesigning the sweeps

The biggest improvement was to the data. In the older sweeps all parameters change together, so each sweep is one line through the space of conditions (Figure 11a) and the method cannot separate, say, steer from speed. I replaced them with space-filling designs (scrambled Sobol sequences; Sobol', 1967) in which ride heights, roll, yaw and corner radius vary independently; steer and speed follow the team's cornering curve and roll is limited at low ride height. All new cases use RRF (straight-line cases with a 500 m radius). Two batches of 32 cases and 16 held-out cases drawn by Latin hypercube sampling (McKay, Beckman and Conover, 1979; Appendix C) are generated by `sweeps/generate_design_sweeps.py`, loaded by `cluster/load_rrf_sweeps.py` and tested by `experiments/rrf_validation.py`. Many runs did not write their surface output, as with sweep 50.34, because this workflow is custom-built and less robust than the team's main one. By the deadline 16 of the 32 batch-1 cases (all cornering), 12 batch-2 cases (all straight-line) and one held-out case had returned; duplicate submissions of a case were kept once. Modes (6 for 90 % of the variation; k = 4 IDW mapping) and sensors were chosen from batch 1 alone, and the other 13 cases were used only for testing. With 20 spaced-greedy sensors, leave-one-out on batch 1 gave 0.318 in Cp against 0.553 with no sensors, and the 13 withheld cases gave 0.289 against 0.564 (worst 0.431; greedy 0.318, QR 0.331, random 0.346). The straight-line cases lie outside the cornering-only training set, yet their error (0.298) was no larger than in leave-one-out. Following the Week 5 notes, duplicates were removed before splitting, every choice in this test was fitted on batch 1 only, and leave-one-out is k-fold cross-validation with k equal to the number of cases. Thirteen cases is a small sample, so this supports the layout rather than proving it.

5.5 Learned reconstruction.

As an alternative to the least-squares rebuild, scikit-learn models (MMA3001 Notebook 5.4 settings) were trained to map the k sensor readings to the r POD coefficients, with POD, sensors and scaler refitted in each fold. At 20 sensors (k > r ≈ 13), neither learned model clearly improved on least squares. Leave-one-out gave 0.241 for the MLP and 0.239 for linear regression against 0.245. Straight-to-cornering gave 0.245 and 0.211 against 0.218. At 10 sensors (k < r) the learned models did better (0.264–0.285 against 0.411). Least squares is then underdetermined and returns the minimum-norm coefficients, while the learned maps use correlations between coefficients seen in training. The MLP's result varied with random seed (0.224–0.239 at k = 20; 0.278–0.481 at k = 10), and linear regression had a worse worst case (0.699 against 0.498). Least squares was kept: it is as accurate when k > r, it is deterministic, and it needs no training.

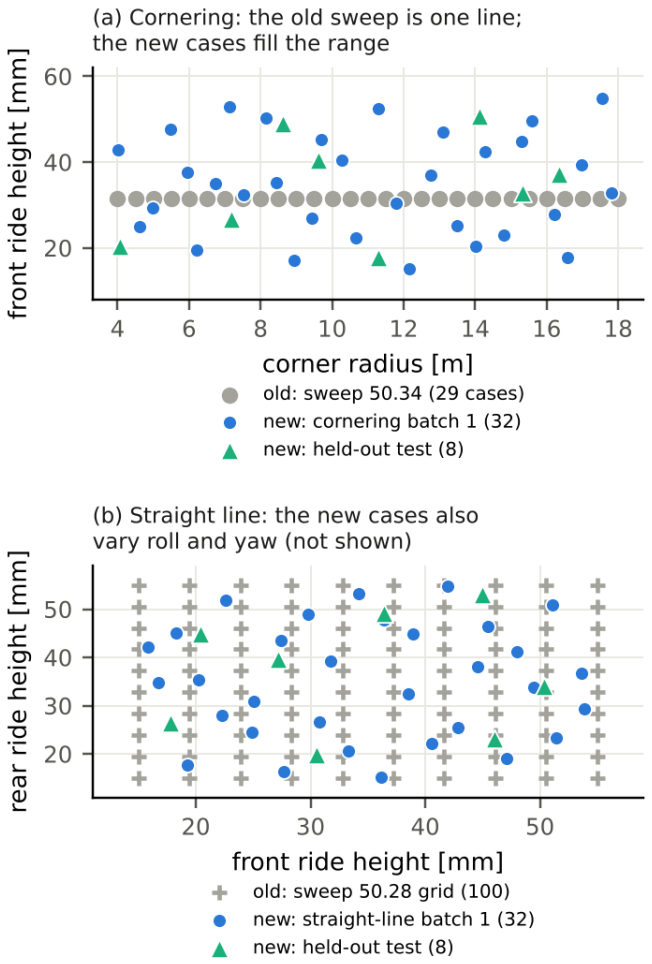


Figure 11. Old sweeps (grey) against the new space-filling design (blue) and the held-out test cases (green).

5.6 Robustness and numerical checks.

Table 7 summarises further checks. They used the straight-to-cornering split unless stated, with k = 20 and POD and sensors refitted per fold. Losing one sensor raised the mean error only slightly (0.218 to 0.229). The worst single loss gave 0.361, and in the worst leave-one-out fold it gave 1.129, above the no-sensor baseline of 0.546. A layout therefore needs redundancy at its most important points. The least-squares residual was tested as a flag for unsupported operating conditions. It flagged none of the 22 cornering cases, which were rebuilt no worse than held-out straight cases, but it tracked the true error (Spearman 0.69). Error away from the sensors cannot appear in the residual, so it is a confidence indicator, not a test of validity. Propagating $\sigma = 0.03$ reading noise through the least-squares solve predicted the added error as $\sigma\|U_s^+\|_F/\sqrt{n}$. This matched Monte Carlo to within 0.92–1.07 for k = 4–80. Greedy placement kept $\text{cond}(U_s) \le 3.3$ and the noise error ≤ 0.017 for every k. At k = 13, just below r, random layouts reached $\text{cond}(U_s) = 132$ and a noise error of 1.32. Sensor noise is therefore not what limits accuracy; the 0.218 error comes from the modes and the data. Computing POD from the Gram matrix squares the condition number, but the retained modes are well conditioned, so it agreed with the full SVD to 4×10^{-14} while running 6 to 7.5 times faster. Storing X in float32 picked the same sensors and halved its memory. Cutting the greedy candidates to 20,000 left the error unchanged (0.2185 against 0.2184), and POD time scaled about linearly with point count (slope 0.95 to 1.10).

Table 7. Robustness and numerical checks (k = 20; straight → cornering unless stated). RMS error in Cp.

Check	Result	Implication
Single-sensor loss	Mean 0.218 → 0.229; worst 0.361; worst LOO fold 1.129	Redundancy needed at key sensors
Residual as out-of-range flag	0 of 22 cornering cases flagged; Spearman 0.69	Confidence score, not a validity test
Noise propagation (σ = 0.03)	Prediction within 0.92–1.07 of Monte Carlo; greedy κ ≤ 3.3, noise ≤ 0.017; random at k = 13: κ = 132, 1.32	Noise is not the limiting error
Gram vs SVD POD	Error ≤ 3.7e-14; same sensors; 0.14 vs 1.06 s; 216 vs 325 MB	Gram method kept
float32 vs float64	Same sensors, RMS 0.2184; 69 vs 138 MB	float32 is fine for storage
Candidate subset (5k → 141,679)	RMS 0.226 → 0.218; greedy 0.02 → 1.3 s	Resolution is not a bottleneck
Drilling position error δ (feasible layout)	Mean error +3 % up to 5 mm; +7–11 % at 7.5 mm; +34–80 % at 10 mm	±5 mm tolerance; locate taps 1 and 3 from a jig

Drilling tolerance. The model assumes each tap reads the pressure at its intended point. To test a mis-drilled tap, every sensor of the feasible layout (Section 5.2) was moved δ mm in a random direction along the surface, the reading was taken from the nearest CFD point there, and the field was rebuilt with the intended mode values (30 trials per δ; `experiments/drilling_tolerance.py`). Up to 5 mm the mean error rose by less than 4 % (leave-one-out 0.252 to 0.261; straight-to-cornering 0.232 to 0.231). At 7.5 mm it rose by 7–11 % and at 10 mm by 34–80 % (Figure D2). The sensitivity is concentrated at a few taps: within 5 mm of sensors 3 and 1, both near the 15 mm edge margin, Cp changes by up to 1.5 and 1.1. A drilling tolerance of ±5 mm is therefore adequate, and those two taps should be located with a jig or from the CAD (Appendix D). The test cannot resolve offsets below the mesh spacing (median 1.5 mm), and the layout was chosen with all cases, so absolute errors are slightly optimistic; the result is the increase with δ.

6. Limitations and lessons learned

- **A result that looks too good needs checking.** One mode and an error of 0.0036 came from fields made nearly constant by a units mismatch (mm grid, m data). Comparing two ensembles built from the same cases exposed it; a single plot of one case's range would have caught it weeks earlier.
- **Combining simulations is part of the model.** Two nearest-neighbour mappings of the same 122 cases gave 13 and 27 modes and opposite conclusions; a mirroring condition never ran. Each combined file now needs a record of the script, mapping method and reference mesh.
- **Sensors must at least match the patterns.** With fewer sensors than modes, least squares cannot fix the pattern amounts and the error stays near the baseline; the count should be chosen from the number of modes and the scanner noise (Section 4.3).
- **Spread the sensors.** Greedy and QR place sensors within millimetres of each other,may cause structural issues iftoo many holes were drilled next to each other; a 0.05 m spacing helped in every test.

- **Protect raw data.** The 50.34 surface files were deleted, so the mapping errors cannot be corrected; raw outputs need a backup before analysis.
- **Next steps:** test on the full held-out set once its runs finish, choose the tap count from the noise study, restrict candidates to the exclusion-free left side, and try a ridge or Gaussian-process regression of the pattern amounts once more cases exist.

7. AI-use reflection

Tools and uses. Throughout this project (August to October 2026), two primary AI tools were utilized: Anthropic's Claude 3.5 Sonnet (including Claude Code) and Google's Gemini 1.5 Pro and 2.5. AI assistance was engaged across several key stages, including scaffolding the initial pipeline architecture, drafting baseline Python scripts, debugging advanced algorithms beyond my immediate syntax familiarity, setting up the Git repository structure, reviewing diagnostic plots, and generating draft text and proofreading for the report, explanatory figures, and toy models. Overall, AI tools generated the initial drafts for approximately 85% of the codebase and 80% of the report text. Using AI served as a high-speed syntax generator and rapid-prototyping accelerator, enabling the rapid exploration and implementation of numerical concepts and data workflows in minutes that would have otherwise required weeks of manual coding.

Checking. To fulfill Monash University and Engineers Australia standards of professional accountability, all AI-generated outputs underwent a rigorous independent verification protocol. Every script and automated test suite was executed independently in the local environment to verify error-free operation. Algorithmic equivalence was confirmed by checking that vectorized matrix searches and explicit loop-based routines selected identical optimal sensor configurations. Regression testing was performed to ensure that the refactored analysis.py module reproduced baseline benchmark outputs exactly. Furthermore, code forensics were conducted on the cluster loader to verify its actual data transformation mechanics rather than relying on AI-generated documentation. Finally, principles of vehicle dynamics and benchmark CFD data were applied to inspect Figures 1 through 3, verifying that pressure fields, flow symmetry, and ride height boundaries were physically plausible, while cross-checking results with teammates ensured overall feasibility.

Decisions that stayed with me: While AI assisted significantly with code syntax and drafting, all strategic engineering choices, scope boundaries, and project directions remained entirely my responsibility. For example, an AI recommendation to train a Deep Neural Network was explicitly rejected due to dataset size limitations, opting instead for a more defensible regularized machine learning model. The parameter sweep bounds and evaluation ranges were also established independently. Furthermore, placement comparisons and ensemble limitations were reported transparently and honestly, rather than selectively framing flawed metrics as unqualified successes.

Errors and unhelpful suggestions. (1) Independent auditing revealed several critical errors and unhelpful suggestions in the AI-generated work. First, the AI-generated documentation claimed that Ensemble B was aligned using inverse-distance weighting (IDW), whereas manual code inspection revealed the cluster loader actually executed nearest-neighbor interpolation, invalidating earlier analytical conclusions. Second, the AI's data mirroring function incorrectly assumed that input data was defined strictly for $y \geq 0$, introducing edge artifacts. Third, the AI attempted to compare experimental runs across completely different datasets as if only the interpolation method varied. Fourth, it proposed ride height sweeps of 20 mm, 31.5 mm, and 45 mm that violated the actual physical operating range of 15 mm to 55 mm, while also inverting plot view titles. Fifth, the AI accepted an unphysical single-mode result on Ensemble A0 and helped build headline report conclusions around it, an underlying units error that was only caught during manual cross-comparison between Ensembles A0 and A. Lastly, the AI consistently exhibited unwarranted certainty, asserting that unverified file sections contained "real experimental data" before checking, which prompted the reclassification of all ungrounded AI claims as unproven hypotheses.

Effect on understanding. Although AI accelerated project development, relying on automated code generation initially introduced knowledge gaps where code was executed without complete prior understanding. To ensure genuine technical mastery, key mathematical and structural concepts—such as how cases are stacked in the data matrix and how leave-one-out cross-validation functions—were re-derived and written manually to guarantee that Sections 1 and 3 could be defended without AI assistance. This reflection established a critical auditor mindset, reinforcing that while AI is a powerful productivity accelerator, the human engineer retains sole professional accountability for the physical truth, safety, and validity of the final engineering work.

Acknowledgements

Lachlan Sayer and I (Monash Motorsport) wrote the original prototype and advised on the sensor constraints, and Mark Thiel ran the CFD sweeps on the M3 cluster. Both agreed to their material being used in this project.

8. References

Berkooz, G., Holmes, P. and Lumley, J. L. (1993) The proper orthogonal decomposition in the analysis of turbulent flows. *Annual Review of Fluid Mechanics*, 25, 539–575.

Drmač, Z. and Gugercin, S. (2016) A new selection operator for the discrete empirical interpolation method: improved a priori error bound and extensions. *SIAM Journal on Scientific Computing*, 38(2), A631–A648.

Fedorov, V. V. (1972) *Theory of Optimal Experiments*. New York: Academic Press.

Harris, C. R. et al. (2020) Array programming with NumPy. *Nature*, 585, 357–362.

Inoba, R., Uchida, K., Iwasaki, Y., Nagata, T., Ozawa, Y., Saito, Y., Nonomura, T. and Asai, K. (2022) Optimization of sparse sensor placement for estimation of wind direction and surface pressure distribution using time-averaged pressure-sensitive paint data on automobile model. *Journal of Wind Engineering and Industrial Aerodynamics*, 227, 105043.

Joshi, S. and Boyd, S. (2009) Sensor selection via convex optimization. *IEEE Transactions on Signal Processing*, 57(2), 451–462.

Katz, J. (2006) Aerodynamics of race cars. *Annual Review of Fluid Mechanics*, 38, 27–63.

Manohar, K., Brunton, B. W., Kutz, J. N. and Brunton, S. L. (2018) Data-driven sparse sensor placement for reconstruction. *IEEE Control Systems Magazine*, 38(3), 63–86.

McKay, M. D., Beckman, R. J. and Conover, W. J. (1979) A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2), 239–245.

Milliken, W. F. and Milliken, D. L. (1995) *Race Car Vehicle Dynamics*. Warrendale, PA: SAE International.

Monash University (2026) *MMA3001 Numerical Methods and Machine Learning: unit notes*. Notebook 2.1 (optimisation, profiling and machine performance), Notebook 2.3 (systems of linear equations, singular values and least squares), Week 3 (interpolation), Notebook 5.5 (evaluating regression performance) and Notebook 9.1 (stability). Clayton: Monash University.

Pauly, M., Gross, M. and Kobbelt, L. P. (2002) Efficient simplification of point-sampled surfaces. *Proceedings of IEEE Visualization 2002*, 163–170.

Sayer, L. (2026) Sparse sensor optimisation: optimising sensor placement using math. Monash Motorsport, unpublished guide and prototype code (analysis.py).

Sayer, L. (n.d.) Wind tunnel testing of Formula Student vehicles. Department of Mechanical and Aerospace Engineering, Monash University, unpublished report.

Shepard, D. (1968) A two-dimensional interpolation function for irregularly-spaced data. *Proceedings of the 23rd ACM National Conference*, 517–524.

Sherman, J. and Morrison, W. J. (1950) Adjustment of an inverse matrix corresponding to a change in one element of a given matrix. *Annals of Mathematical Statistics*, 21(1), 124–127.

Sirovich, L. (1987) Turbulence and the dynamics of coherent structures, Part I: coherent structures. *Quarterly of Applied Mathematics*, 45(3), 561–571.

Sobol', I. M. (1967) On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4), 86–112.

Thiel, M. (2026) Aeromap CFD sweeps 50.27 to 50.42. Monash Motorsport, M3 cluster, unpublished data.

Thomas, A. J., Hazon, J. E. and Deva Doss, C. P. (n.d.) CFD correlation of wind tunnel testing on a Formula-SAE car. Department of Mechanical and Aerospace Engineering, Monash University, unpublished report.

Virtanen, P. et al. (2020) SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

Zhang, X., Toet, W. and Zerihan, J. (2006) Ground effect aerodynamics of race cars. *Applied Mechanics Reviews*, 59(1), 33–49.

Appendix A. Glossary

Term	Meaning in this report
Attitude	How the car sits relative to the road and the air: ride heights, pitch, roll and yaw.
Case	One CFD simulation at one set of conditions.
CFD	Computational fluid dynamics: solving the equations of air flow numerically on a mesh.
Corner radius R	Radius of the circular path the car follows in a turn (m).
Cp	Pressure coefficient, $(p - p\infty)/(\frac{1}{2}\rho V^2)$: 0 = surrounding pressure, +1 = air at rest, negative = suction.
D-optimal	Choosing measurement points that maximise the determinant of the information matrix, i.e. that pin down the unknowns most tightly against independent noise.
Diffuser	Expanding passage that slows air down and raises its pressure.
Downforce	Aerodynamic force pressing the car onto the road.
Ensemble	A set of cases analysed together.
FRH, RRH	Front and rear ride height: floor-to-ground gap near the front and rear axles (mm).
Ground effect	Extra suction produced when a shaped body runs close to the ground.
IDW	Inverse-distance weighting: interpolation that averages nearby values, weighting each by 1/distance.
Lateral acceleration	Sideways acceleration in a turn, V^2/R, often quoted in g (9.81 m/s²).
Leave-one-out (LOO)	Validation in which each case is hidden in turn and predicted from the others.
Mesh	The set of small cells, and their points, on which CFD computes the flow.
Mode	One of the building-block pressure patterns found by POD.
Pitch	Nose-up or nose-down rotation of the car (FRH ≠ RRH).
POD	Proper orthogonal decomposition: finding the dominant patterns in a set of fields; computed here with the SVD.
Pressure tap	Small hole in the surface connected by a tube to a pressure sensor; one "sensor" in this report.
QR pivoting	Ordering candidate points by a pivoted QR factorisation of U_r^T; a standard sensor-selection method.
RMS error	Root mean square of the point-by-point differences between the rebuilt and the CFD field.
Roll	Sideways lean of the car body, mainly in corners.
RRF	Rotating reference frame: simulation set-up for cornering in which the air approaches along curved paths.
Steer	Angle of the front wheels relative to the car.
SVD	Singular value decomposition, $X = U\Sigma V^T$: splits a matrix into patterns (U), their importance (Σ) and their amounts (V).
Sweep	A set of cases run together that vary one or more conditions, e.g. sweep 50.28.
SYM	Straight-line simulation of half the car with a mirror plane on the centreline.
Yaw	Angle between where the car points and the direction it travels (or the direction the air comes from).

Appendix B. Reproducing the results

Layout: `sensor_opt/core.py` (POD, greedy placement, reconstruction, `CfdDataError`), `analysis.py` (main pipeline), `cluster/` (loaders run on M3), `sweeps/` (sweep generators, case files, design plots), `experiments/` (toy model, schematics, validation extras), `CAD/point_sorter.py` (exclusion mask), `tests/`, `docs/` (pytest report, HTML API docs, AI log). Raw CFD data and ensemble files are not committed (see `.gitignore`); place them in `data/` or set `SSP_DATA_DIR`.

- Environment: `python -m venv .venv`, activate it, then `pip install numpy scipy pandas matplotlib plotly pytest pdoc`.
- Pipeline: set `SSP_DATA_DIR=data` and `SSP_ENSEMBLE=<ensemble>.npz`, then `python analysis.py`.
- Tests and documentation: `pytest -v` (uses `pytest.ini`); `pdoc -o docs/html sensor_opt`.
- Extra validation numbers: `python experiments/validation_extras.py data/<ensemble>.npz --params data/case_params.csv`.
- Cornering tests (Table 4): `python experiments/cornering_test.py data/side_diffuser_ensemble_highres_NN.npz --params data/case_params.csv` (B1) and the same with `side_diffuser_ensemble.npz` (B2); Figure 8: `python experiments/plot_cornering_report.py <B1 csv> <B2 csv>`. Sensor count and noise (Figure 7): `python experiments/sensor_count_ensemble.py <B1 npz> --params data/case_params.csv`. Placement on A (Table 5): `results/placement_comparison_highres.txt`.
- Figures: `python experiments/toy_diffuser_demo.py` (Figures 2, 5), `python experiments/vehicle_schematics.py` (Figure 1), `python sweeps/plot_design_coverage.py` (Figure 11, Appendix C).
- Sweep case files: `python sweeps/generate_design_sweeps.py` (writes `sweeps/new/`).

Appendix C. Full coverage of the new sweep designs

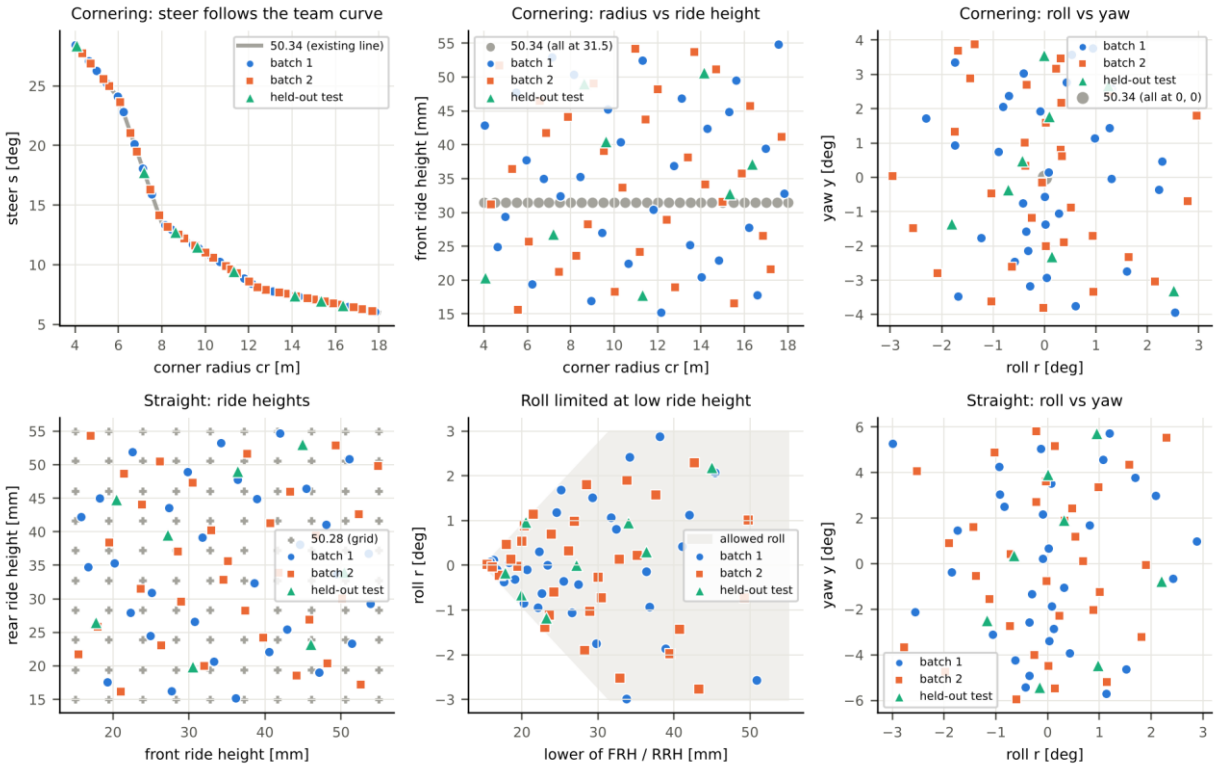


Figure C1. All six projections of the new designs: batch 1 (blue), batch 2 (orange), held-out test cases (green) and the older sweeps (grey). Roll is limited where the ride height is low.

Appendix D. Sensor layout views and CAD export

Figure D1 shows the feasible 20-sensor layout from four directions. The same points are in `results/k20/feasible_sensors_NX_k20.txt` (one line per sensor, X Y Z in mm, CFD coordinates), which Siemens NX imports with File → Import → Points from File; `results/k20/feasible_layout.csv` gives the sensor numbers, edge distances and spacings. Every 20-sensor result is rerun with the environment variable `SENSORS_K=20` (default 24) and written to `results/k20/`; for example `python experiments/drilling_tolerance.py data/side_diffuser_ensemble_highres_NN.npz --params data/case_params.csv --layout results/k20/feasible_layout.csv --outdir results/k20` (Figure D2).

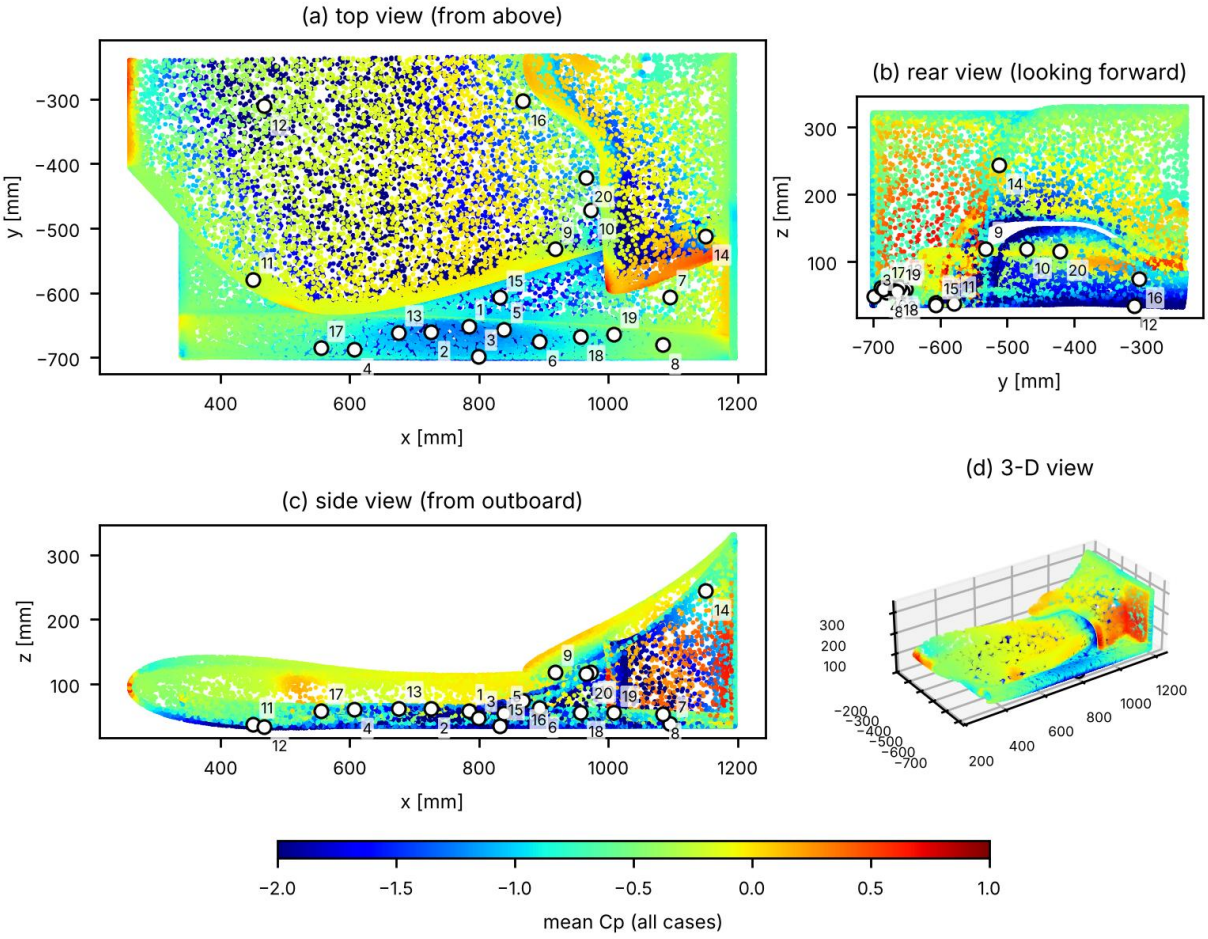


Figure D1. Feasible layout (white circles, numbered as in `results/k20/feasible_layout.csv`) on the mean Cp of all 122 cases: (a) from above, (b) from the rear, (c) from outboard, (d) 3-D.

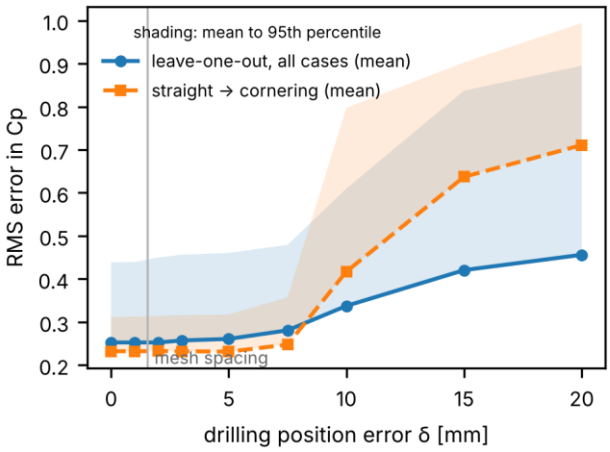


Figure D2. Reconstruction error against drilling position error for the feasible 20-sensor layout (30 random trials per δ). Lines are means; shading spans the mean to the 95th percentile. Below the mesh spacing (grey line) most moved taps fall back on the same CFD point.